

LN8-2. 梯度下降

张晓今

计算机学院数据挖掘与机器学习实验室

华中科技大学 Hopcroft 计算科学研究中心



目录

- 1 介绍
 - 为什么使用梯度下降
 - 梯度下降
- 2 批量梯度下降 (BGD)
 - 批量梯度下降 (BGD)
 - 例子
 - 总结
- 3 随机梯度下降 (SGD)
 - 随机梯度下降 (SGD)
 - Example
 - 总结
- 4 小批量随机梯度下降
 - 小批量随机梯度下降
 - 对比 BGD&SGD
 - 如何设置小批量梯度下降
 - 例子
 - 总结

目录

- 1 介绍
 - 为什么使用梯度下降
 - 梯度下降
- 2 批量梯度下降 (BGD)
 - 批量梯度下降 (BGD)
 - 例子
 - 总结
- 3 随机梯度下降 (SGD)
 - 随机梯度下降 (SGD)
 - Example
 - 总结
- 4 小批量随机梯度下降
 - 小批量随机梯度下降
 - 对比 BGD&SGD
 - 如何设置小批量梯度下降
 - 例子
 - 总结

为什么使用梯度下降

基本思想

训练一个网络的过程，实际上是对网络的数十亿个参数进行优化。我们需要一个合适的办法最小化凸连续可微损失函数 $\downarrow(w)$

- 这个损失函数的值给了一个度量方法，表示网络在给定数据集上的性能离最优还有多少差距。
- 当我们在开始时随机初始化权重时，这个损失函数的值通常是非常大的。
- 我们需要优化损失函数，使网络获得更好的性能。

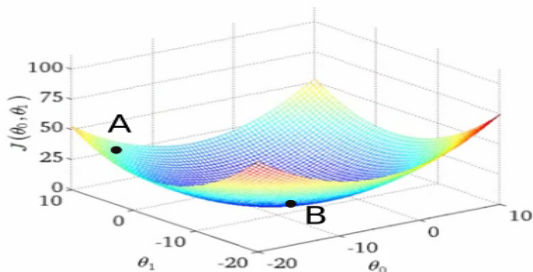
梯度下降

机器学习中，梯度下降是一种最小化损失函数的优化算法，被描述为：

```
while True:
```

```
weights_grad = evaluate_gradient(loss_fun,data,weights)
```

```
weights+=-stepsize*weights_grad
```



Gradient Descent

梯度下降

假设 $f(\mathbf{x})$ 是一个连续可微函数。对于 ϵ ，其值足够小，根据泰勒展开公式，我们得到以下近似：

$$f(\mathbf{x} + \epsilon) \approx f(\mathbf{x}) + \epsilon \mathbf{f}'(\mathbf{x})$$

一维函数的梯度是一个标量，也称为导数。接下来，找到一个常数 $\eta > 0$ ，使 $|\eta \mathbf{f}'(\mathbf{x})|$ 足够小，从而将 ϵ 替换为 $-\eta \mathbf{f}'(\mathbf{x})$ 并且得到：

$$f(\mathbf{x} - \eta \mathbf{f}'(\mathbf{x})) \approx f(\mathbf{x}) - \eta \mathbf{f}'(\mathbf{x})^2$$

如果导数 $\mathbf{f}'(\mathbf{x}) \neq 0$ ，那么 $\eta \mathbf{f}'(\mathbf{x})^2 > 0$ ，因此

$$f(\mathbf{x} - \eta \mathbf{f}'(\mathbf{x})) \leq f(\mathbf{x})$$

这意味着，如果我们使用

$$\mathbf{x} \leftarrow \mathbf{x} - \eta \mathbf{f}'(\mathbf{x})$$

来迭代 $\mathbf{x}$ ，函数值 $f(\mathbf{x})$ 会下降。

梯度下降

上述梯度下降算法中的 η 通常称为学习率。这是一个超参数，需要手动设置。如果我们使用的学习率太小这将导致 x 以非常慢的速度更新，需要更多轮的迭代才能得到更好的解。假设 $f(x) = x^2$ ， $\eta = 0.05$ ，迭代步数 $= 10$

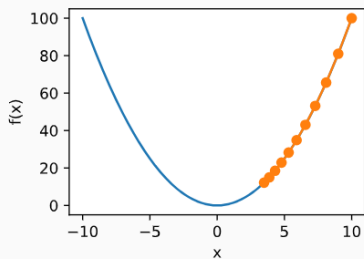


图: $\eta=0.05$

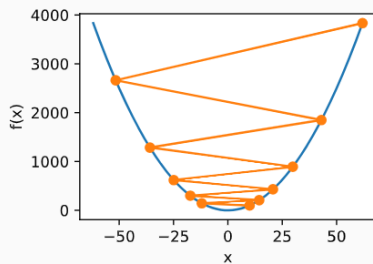


图: $\eta=1.1$

目录

- 1 介绍
 - 为什么使用梯度下降
 - 梯度下降
- 2 批量梯度下降 (BGD)
 - 批量梯度下降 (BGD)
 - 例子
 - 总结
- 3 随机梯度下降 (SGD)
 - 随机梯度下降 (SGD)
 - Example
 - 总结
- 4 小批量随机梯度下降
 - 小批量随机梯度下降
 - 对比 BGD&SGD
 - 如何设置小批量梯度下降
 - 例子
 - 总结

批量梯度下降 (BGD)

基本思想：

在计算整个训练集的梯度误差之后，再对参数进行更新。

例子

test-BGD.py, 尝试设置不同的学习率来查看输出:

<https://github.com/IEC-lab/MachineLearning2019/blob/master/GD/test-BGD.py>

对 BGD 的总结

好处:

- 对模型的更新较少意味着这种梯度下降的变体比随机梯度下降的计算效率更高。
- 降低更新频率会使误差梯度更稳定，并可能在某些问题上收敛的更稳定。
- 预测误差计算和模型更新分离使该算法能够基于并行处理实现。

对 BGD 的总结

弊端：

- 更稳定的梯度可能导致模型过早收敛到一个不太优的参数。
- 训练阶段结束时，更新需要在所有训练样本中计算累积误差，而这存在额外复杂性。
- 批量梯度下降以这样一种方式实现，需要整个训练数据集在内存中并且可供算法使用。
- 对于大型数据集，模型更新以及相应的训练速度可能会变得非常缓慢。

目录

- 1 介绍
 - 为什么使用梯度下降
 - 梯度下降
- 2 批量梯度下降 (BGD)
 - 批量梯度下降 (BGD)
 - 例子
 - 总结
- 3 随机梯度下降 (SGD)
 - 随机梯度下降 (SGD)
 - Example
 - 总结
- 4 小批量随机梯度下降
 - 小批量随机梯度下降
 - 对比 BGD&SGD
 - 如何设置小批量梯度下降
 - 例子
 - 总结

随机梯度下降

测试

在机器学习中，训练集是巨大的，没有简单的公式存在，评估梯度的求和会产生巨大的开销，因为评估梯度需要评估所有求和函数的梯度。如何最小化每次迭代的计算成本？

随机梯度下降

解决方法

每一步都对训练集的一个子集进行采样，而不是对整个训练集进行采样。参数在计算单个训练样本的误差梯度后更新。

在伪代码中，随机梯度下降可以表示为：

- 选择参数 w 和学习率 η 的初始向量
- 重复操作，直到获得一个近似的最小值：
 - 随机洗牌训练集中的例子。
 - $i=1, 2, \dots, n$, 执行：
 - $w = w - \eta f_i'(x)$

例子

我们可以在梯度中添加均值为 0 的随机噪声来模拟 SGD:

sgd.py

```
import sys
import math
import numpy as np

def train_2d(trainer):
    x1, x2, s1, s2 = -5, -2, 0, 0
    results = [(x1, x2)]
    for i in range(20):
        x1, x2, s1, s2 = trainer(x1, x2, s1, s2)
        results.append((x1, x2))
    print('epoch %d, x1 %f, x2 %f' % (i + 1, x1, x2))
    return results

def sgd_2d(x1, x2, s1, s2):
    return (x1 - eta * (2 * x1 + np.random.normal(0.1)),
            x2 - eta * (4 * x2 + np.random.normal(0.1)), 0, 0)

train_2d(sgd_2d)
```

Example

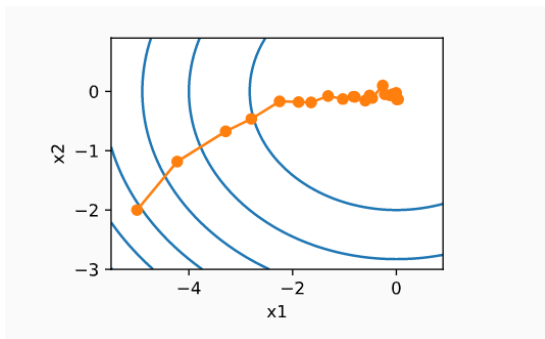


图: epoch=20

可以看出，在 SGD 中自变量的迭代轨迹比在梯度下降中更为曲折。这是由于实验中加入了噪声，降低了模拟随机梯度的准确性。在实际中，这种噪声通常来自训练数据集中的单个样本。

例子

test-SGD.py:

<https://github.com/IEC-lab/MachineLearning2019/blob/master/GD/test-SGD.py>

对 SGD 的总结

好处:

- 频繁地更新使我们可以立即观察到模型性能和改进的速度。
- 对初学者来说, 这种梯度下降的变体可能是最容易理解和实现的。
- 模型更新频率的增加使得某些问题上可以更快地学习。
- 有噪声的更新过程可以让模型避免局部最小值 (例如过早收敛)。

对 SGD 的总结

弊端：

- 频繁地更新模型比其他梯度下降方法计算成本更高，在大型数据集上训练模型需要更长的时间。
- 频繁地更新会导致梯度信号存在噪声，这可能会导致模型参数和模型误差的震荡（在训练周期上有更高的方差）。
- 梯度噪声学习过程会使算法很难确定模型的误差最小值。

目录

- 1 介绍
 - 为什么使用梯度下降
 - 梯度下降
- 2 批量梯度下降 (BGD)
 - 批量梯度下降 (BGD)
 - 例子
 - 总结
- 3 随机梯度下降 (SGD)
 - 随机梯度下降 (SGD)
 - Example
 - 总结
- 4 小批量随机梯度下降
 - 小批量随机梯度下降
 - 对比 BGD&SGD
 - 如何设置小批量梯度下降
 - 例子
 - 总结

小批量随机梯度下降

基本思想：

小批量梯度下降试图在随机梯度下降的稳健性和批量梯度下降的效率之间找到一个平衡。

在训练集的一个子集上，计算误差梯度来进行参数的更新。

小批量随机梯度下降

在伪代码中，小批量随机梯度下降可以表示为：

令 θ = 模型参数， max_iters = epoch 的轮数

对于 $itr = 1, 2, 3, \dots, max_iters$:

for $mini_batch(X_mini, y_mini)$:

- Forward Pass on the batch X_mini :

- Make predictions on the mini-batch
- Compute error in predictions $J(\theta)$ with the current values of the parameters

- Backward Pass:

- Compute $gradient(\theta)$ = partial derivative of $J(\theta)$ w.r.t. θ

- Update parameters:

- $\theta = \theta - learning_rate * gradient(\theta)$

对比 BGD&SGD

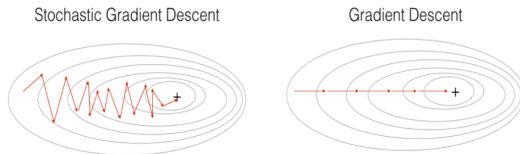


图: SGD vs BGD

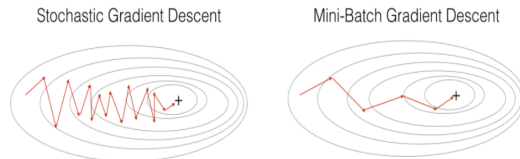


图: SGD vs MSGD

对比 BGD&SGD

总之，梯度下降、小批量梯度下降和随机梯度下降之间的区别在于执行一个更新步骤所使用的样本数量。通过调整的 Batch size 的大小，它优于梯度下降或随机梯度下降。

Mini-batch sizes, 通常称为“批处理大小”, 通常针对执行实现的计算体系结构的某个方面进行调优。例如 2 的幂, 适合 GPU 或 CPU 硬件的内存要求, 如 32、64、128、256 等。

批处理大小是学习过程中的一个重要参数。

- 小的批处理大小使学习过程以训练过程中的噪声为代价快速收敛。
- 大的批处理大小使学习过程缓慢收敛, 但准确估计了误差梯度。

设置批处理大小

Tip1:

- 批处理大小的默认值可能是 32。
-Revisiting Small Batch Training for Deep Neural Networks, 2018.

Tip2:

- 当调整批处理大小时，回顾模型训练时验证误差的学习曲线是一个好办法。

Tip3:

- 在调优所有其他超参数后，调优批处理大小和学习率。

例子

test-MSGD.py:

<https://github.com/IEC-lab/MachineLearning2019/blob/master/GD/test-MSGD.py>

对 MSGD 总结

好处：

- 模型更新频率高于批量梯度下降，允许更鲁棒的收敛，避免局部最小值。
- 批量更新提供了一个计算上比随机梯度下降更有效的过程。
- 批处理既保证在内存中无所有训练数据的效率，也保证了算法效率。

对 MSGD 总结

弊端：

Mini-batch 需要为学习算法设置一个额外的“Mini-batch size”超参数。错误信息必须在小批量的训练样本中积累，例如批量梯度下降。

The end!